

WEEK 8 PRACTICAL

Task1: Write a C program to demonstrate dynamic memory allocation using malloc(), calloc(), realloc(), and free(). Also identify and analyze memory leaks using Valgrind. Aim

Program

```
GNU nano 7.2 dynamic_memory.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;

    // malloc
    p = malloc(2 * sizeof(int));
    p[0] = 10;
    p[1] = 20;

    printf("After malloc: %d %d\n", p[0], p[1]);

    // realloc
    p = realloc(p, 3 * sizeof(int));
    p[2] = 30;

    printf("After realloc: %d %d %d\n", p[0], p[1], p[2]);

    // free
    free(p);

    // calloc
    p = calloc(2, sizeof(int));

    printf("After calloc: %d %d\n", p[0], p[1]);

    // Intentionally not freeing p
    // This creates a memory leak

    return 0;
}
```

```
saicharan_reddi@SAICHARANREDDY:~$ nano memory_maps.c
saicharan_reddi@SAICHARANREDDY:~$ gcc memory_maps.c -o memory_maps
saicharan_reddi@SAICHARANREDDY:~$ ./memory_maps
Process ID: 613
Global variable address: 0x5bab1ae54010
Stack variable address: 0x7ffc6ca630d4

Process is running...
Open another terminal and execute:
cat /proc/613/maps

Press Enter to terminate...

saicharan_reddi@SAICHARANREDDY:~$ cat /proc/1722/map
cat: /proc/1722/map: No such file or directory
saicharan_reddi@SAICHARANREDDY:~$ pwd
/home/saicharan_reddi
saicharan_reddi@SAICHARANREDDY:~$ nano dynamic_memory.c
saicharan_reddi@SAICHARANREDDY:~$ gcc dynamic_memory.c -o dynamic_memory
saicharan_reddi@SAICHARANREDDY:~$ ./dynamic_memory
After malloc: 10 20
After realloc: 10 20 30
After calloc: 0 0
saicharan_reddi@SAICHARANREDDY:~$ |
```

Output

Observation

The program successfully demonstrated dynamic memory allocation using malloc(), calloc(), realloc(), and free(). malloc() allocated memory for integers, realloc() increased the allocated memory size, and calloc() allocated memory initialized to zero. Valgrind detected an **8-byte memory leak** because the memory allocated using calloc() was not freed. After using free() properly, the allocated memory was released successfully.

Task2: Write a C program to demonstrate the Copy-on-Write (COW) mechanism after a fork() system call. Show that the parent and child processes have separate copies of the data when the child modifies it.

Program

```

GNU nano 7.2                                copy_on_write.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int data = 10;

    printf("Before fork():\n");
    printf("Parent data: %d\n", data);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("\nChild process:\n");
        printf("Before modification: %d\n", data);

        data = 99;

        printf("After modification: %d\n", data);
    }
    else
    {
        wait(NULL);

        printf("\nParent process:\n");
        printf("Parent data: %d\n", data);
    }

    return 0;
}

```

Output

```

saicharan_reddi@SAICHARANREDDY:~$ nano copy_on_write.c
saicharan_reddi@SAICHARANREDDY:~$ gcc copy_on_write.c -o copy_on_write
saicharan_reddi@SAICHARANREDDY:~$ ./copy_on_write
Before fork():
Parent data: 10

Child process:
Before modification: 10
After modification: 99

Parent process:
Parent data: 10
saicharan_reddi@SAICHARANREDDY:~$ |

```

Observation

The program demonstrated the Copy-on-Write mechanism using `fork()`. Initially, both the parent and child processes had the same value of the variable. When the child modified the variable, its value changed only in the child process, while the parent retained the original value. This shows that the operating system provides separate copies of the data when a modification occurs.

